

AGENT TYPES

WAFT | Catalog of Base Agent Classes

Overview

WAFT provides several base agent types optimized for different tasks. All agents can be evolved through the Scint Gym.

General Purpose Agents

BaseAgent

The foundation class for all agents.

```
1 class BaseAgent(Agent):
2     """Minimal agent with basic LLM interaction."""
3
4     def process(self, input: str) -> str:
5         return self.llm.generate(self.system_prompt, input)
```

 Python

Best for: Starting point, learning WAFT

ChatAgent

Conversational agent with memory.

```
1 class ChatAgent(Agent):
2     """Agent with conversation history."""
3
4     def process(self, input: str) -> str:
5         self.history.append({"role": "user", "content": input})
6         response = self.llm.chat(self.history)
7         self.history.append({"role": "assistant", "content": response})
8         return response
```

 Python

Best for: Multi-turn conversations, context retention

Specialized Agents

CodeAgent

Agent optimized for code generation and analysis.

Capabilities:

- Code generation with syntax validation
- Code review and refactoring
- Test generation
- Documentation writing

```
1 class CodeAgent(Agent):  
2     """Agent specialized for code tasks."""  
3  
4     system_prompt = """You are an expert programmer.  
5     Write clean, tested, documented code."""  
6  
7     def process(self, input: str) -> str:  
8         response = self.llm.generate(self.system_prompt, input)  
9         # Validate syntax before returning  
10        if self.contains_code(response):  
11            self.validate_syntax(response)  
12        return response
```

 Python

ResearchAgent

Agent for information gathering and synthesis.

Capabilities:

- Source verification
- Citation management
- Fact checking
- Summarization

```
1 class ResearchAgent(Agent):  
2     """Agent specialized for research tasks."""  
3  
4     def process(self, input: str) -> str:  
5         # Gather information  
6         sources = self.search(input)  
7         # Verify and synthesize  
8         verified = self.verify_sources(sources)  
9         return self.synthesize(verified)
```

 Python

Scint-Specialized Agents

StabilizerAgent

Agent designed specifically for Scint stabilization.

Capabilities:

- Scint detection
- Type classification
- Stabilization injection
- Verification

```
1 class StabilizerAgent(Agent):
2     """Agent specialized for Scint stabilization."""
3
4     def process(self, input: str) -> str:
5         # Detect any Scints in input
6         scints = self.detect_scints(input)
7
8         if scints:
9             # Stabilize each Scint
10            for scint in scints:
11                input = self.stabilize(input, scint)
12
13            return self.llm.generate(self.system_prompt, input)
```

 Python

ValidatorAgent

Agent focused on output validation.

```
1 class ValidatorAgent(Agent):
2     """Agent that validates its own output."""
3
4     def process(self, input: str) -> str:
5         response = self.llm.generate(self.system_prompt, input)
6
7         # Self-validate
8         validation = self.validate(response)
9         if not validation.passed:
10            # Retry with feedback
11            response = self.retry_with_feedback(
12                input, response, validation.issues
13            )
14
15            return response
```

 Python

Creating Custom Agents

Extend any base class:

```
1 from waft.agents import CodeAgent
```

 Python

```
2
3 class MyCustomAgent(CodeAgent):
4     """My specialized code agent."""
5
6     system_prompt = """You are MY specialized assistant..."""
7
8     def process(self, input: str) -> str:
9         # Add custom preprocessing
10        input = self.preprocess(input)
11
12        # Call parent
13        result = super().process(input)
14
15        # Add custom postprocessing
16        return self.postprocess(result)
```

WAFt Agent Types | The Building Blocks of Evolution